

Conformance to zkPoP, proved — and what one machine-checked result costs

A certificate that this project implements the zkPoP specification, together with the certified cost ledger of a single accepted result and its Bitcoin counterfactuals

Abstract

This paper presents a proof that the present project follows zkPoP — the self-hosting zero-knowledge proof-of-proof architecture whose specification the project set out to implement — and, on top of it, the certified accounting that the architecture was built to carry: what one machine-checked result costs, what it costs per token once the hardware and the frontier programme are fully amortized, and how much Bitcoin the same energy and the same money would have bought instead. None of this is argued in prose and none of it is established by a table of cross-references. Conformance is a Lean 4 predicate, `ZkPoP.Conformance.Obligations`, whose fields are the specification’s requirements written out with all their hypotheses; the findings are a second predicate, `ZkPoP.Paper.LedgerFindings`, whose fields are the exact rational values and the comparative statics; the fact that the inputs were sealed before the fact is a third, `ZkPoP.Paper.PublishedEvidence`. The single theorem

$$\text{ZkPoP.Paper.paper_certificate} : \text{Obligations } F \ G \ q \wedge \text{AppObligations} \\ \wedge \text{LedgerFindings} \wedge \text{PublishedEvidence}$$

proves all four at once. The proof is machine-checked, contains no `sorry`, and its axiom closure is Lean’s three standard axioms `propext`, `Classical.choice` and `Quot.sound`. The paper states the obligations, reports the findings with the theorem that certifies each one, shows the artifact as it is rendered, records the audit, and — as much of the point — delimits precisely what is *not* asserted: the cryptographic hardness assumptions, the hash idealisations, and every empirical figure, all of which appear as named hypotheses or as replaceable data rather than as theorems.

Contents

1 What is being claimed	2
2 Summary of the findings	3
3 The cost ledger	5
4 The Bitcoin counterfactual	6
5 The protocol-layer obligations	7
6 The application-layer obligations	8
7 The conformance theorem and the paper certificate	9
8 The published proof: sealing the inputs	11

9 The audit: proof that these are proofs	12
10 What is not claimed	13
11 Reading the artifact	14

1 What is being claimed

A specification is followed, or it is not, and the difference is usually settled by reading. The reader compares a document to an implementation and forms a judgement. That judgement is exactly the object this paper replaces.

Three claims are made here, and they are of three different kinds.

1. **Conformance** (§5–§7). The project implements zkPoP. This is a mathematical claim about the project’s own definitions, quantified over arbitrary parameters, and it is a theorem.
2. **Findings** (§2–§4). One accepted machine-checked result consumes 3 360 000 J of grid energy, is charged \$1251.03 fully loaded, costs \$1251.03 per million tokens once the cluster and the frontier programme are amortized, and corresponds to 87.5 satoshi of mined Bitcoin on an equal-energy basis and about 1.466 million satoshi on an equal-money basis. These are theorems *about a parameter set*: they say what follows from the recorded figures, never that the figures are correct.
3. **Evidence** (§8). The parameter set was sealed before the fact. The published Pedersen commitment opens to exactly one eight-digit input vector, that vector is the ledger’s own inputs in stated units, the recorded Σ -protocol transcript verifies, and a transcript simulated with no witness at all verifies exactly as well — so the transcript is evidence of knowledge and not a disclosure of it.

The zkPoP specification asks for an architecture with seven ingredients: a semantic bridge from a typed object language to arithmetic constraints; an exact compilation of polynomial constraints into flat gates; a discipline that keeps index wires honest inside a finite field; a universal checker that can consume its own code word, so that proofs may be nested; a commitment scheme with a Σ -protocol over it; a threshold distribution layer that is private and not merely durable; and a physical artifact — commitment, transcript, catalogue — that an offline decoder can check. Each of these is a mathematical requirement, so each can be written as a proposition, and the conjunction of those propositions can be proved.

That is what `RequestProject/ZkPoPConformance.lean` does. It declares a structure whose every field is one requirement, quantified over arbitrary parameters:

```
Obligations (F : Type*) [Field F] [CharZero F] (G : Type*) (q : ℕ) [AddCommGroup G]
               [Module (ZMod q) G] ...
```

and then proves `project_conforms F G q : Obligations F G q` from the project’s own definitions. `RequestProject/ZkPoPPaper.lean` adds the other two predicates and the single certificate that carries all of them.

Remark 1. *Two properties of this arrangement matter more than the individual proofs. First, the obligations are stated before being met: a field of the structure is a claim about ZkPoP’s definitions, not a restatement of a lemma’s proof. Second, an obligation whose truth depends on an assumption carries that assumption in its own statement — binding of the published artifact, for instance, quantifies over collision-freeness of the Fiat–Shamir hash. A reader who rejects an assumption knows exactly which field to discount, and the remaining fields are unaffected.*

2 Summary of the findings

The unit of accounting is *one accepted machine-checked result*: a 10^6 -token reasoning workload whose output is checked, not an abstract token and not an API invoice. The scenario is a 64-accelerator cluster drawing 56 kW on the IT side at a facility PUE of 1.2, serving 20 000 tokens per second, bought for \$2.4M over a four-year life, on electricity at \$0.08/kWh, running a frontier programme of \$250B amortized over 2×10^{14} deployed lifetime tokens. Every one of those figures lives in `RequestProject/Data/Scenarios.lean` and is replaceable; every number below is a theorem about them.

Quantity	Exact value	$\approx$	Theorem
Grid energy of the run	3 360 000 J	0.933 kWh	<code>claim_energy</code>
in kilowatt hours	14/15 kWh	0.933 kWh	<code>resultEnergyKWh_eq</code>
per token	84/25 J	3.36 J	<code>energyPerToken_eq</code>
Electricity	28/375	\$0.075	<code>electricityUSD_eq</code>
Cluster depreciation	625/657	\$0.951	<code>depreciationUSD_eq</code>
Marginal cost	84257/82125	\$1.026	<code>marginalUSD_eq</code>
Programme allocation	1250	\$1250	<code>amortizedUSD_base</code>
Total investment	102740507/82125	\$1251.03	<code>claim_total_investment</code>
Cost per token	102740507/82125000000	\$1251.03/Mtok	<code>claim_cost_per_token</code>
BTC, same energy	7/8000000	87.5 sats	<code>claim_btc_from_same_energy</code>
BTC, same money	102740507/7008000000	1 466 046 sats	<code>claim_btc_from_same_money</code>

Table 1: The headline ledger of one accepted machine-checked result. All values are exact rationals in Lean; the decimal column is rounded for reading only.

Three facts about this table are themselves theorems, and they are the ones worth carrying away.

- **The sunk cost dominates the physics by more than $1218\times$** (`sunk_cost_dominates_marginal`). Of \$1251.03, exactly \$1250 is the allocated share of the frontier programme and \$1.03 is everything physical — electricity and the cluster. What a result “costs” is therefore overwhelmingly an amortization convention, not a measurement.
- **The lifetime-volume assumption moves the answer by more than $350\times$** (`lifetime_assumption_dominates`). At 10^{16} deployed lifetime tokens the same result costs \$26.03 (`ledgerTotal_high`); at 10^{17} , \$3.53 (`ledgerTotal_veryHigh`). The physics is unchanged in all three rows. The allocation is antitone in lifetime volume in general (`training_cost_scale_monotonicity`), and the crossover at which the programme allocation stops exceeding the marginal cost is 2.44×10^{17} tokens (`amortized_le_marginal_of_large_lifetime`).
- **Discarded search scales the ledger linearly, and leaves the cost per token alone** (`ledgerTotal_scales`, `ledgerCostPerToken_scale_invariant`). A hundredfold discovery multiplier makes the campaign cost \$125 103 (`ledgerTotal_campaign`) while the price of a token is unchanged: what changes is how many tokens one accepted result is charged with.

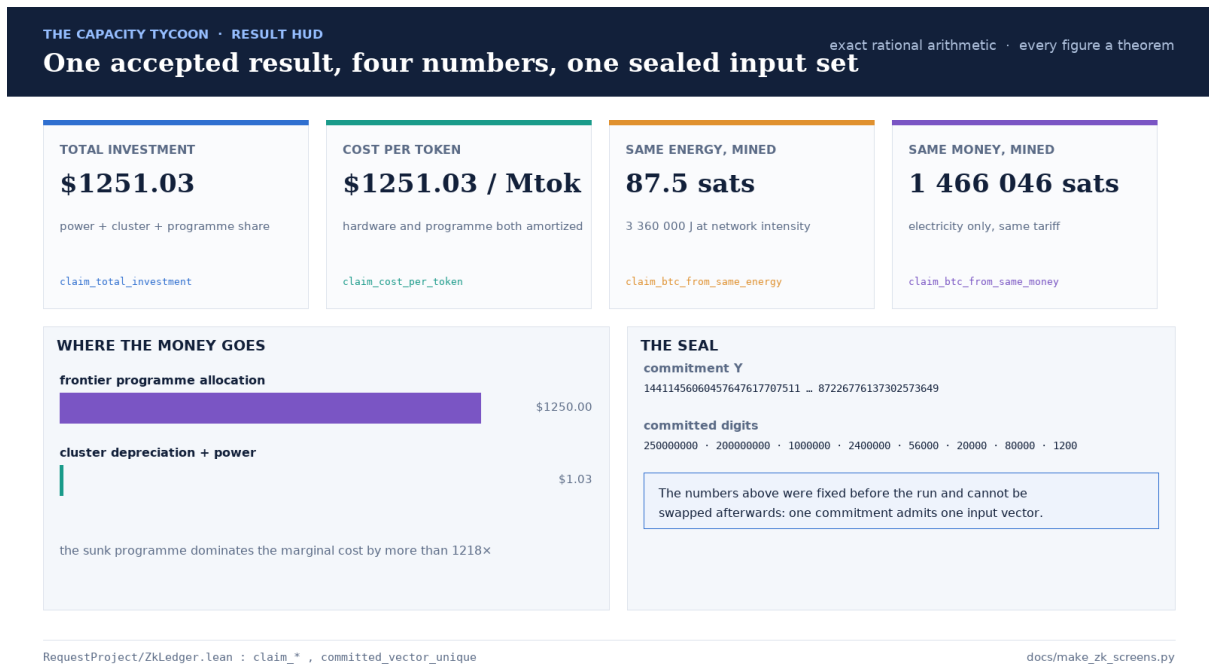


Figure 1: The four headline numbers as the project renders them, each labelled with the theorem that certifies it. Reproduced from `docs/figures/`; regenerated by `docs/make_zk_screens.py`.

What one machine-checked result costs

Fully loaded cost of a 10^6 -token reasoning result on a 64-accelerator cluster, in exact rational arithmetic. Log scale: the three components are three orders of magnitude apart.

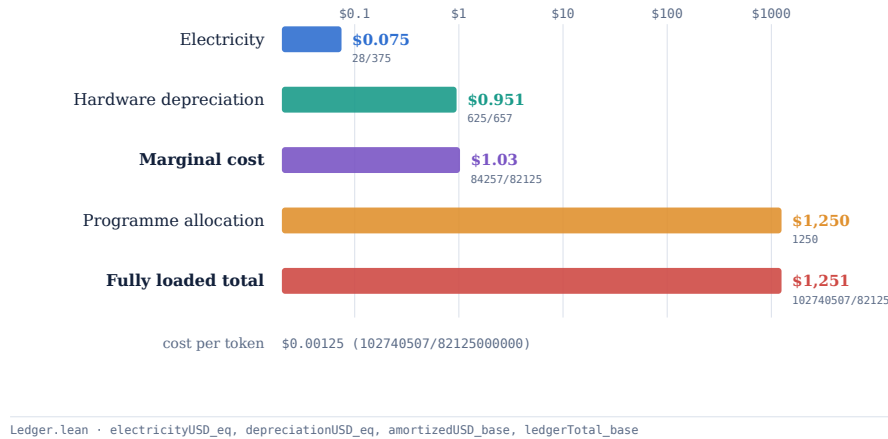
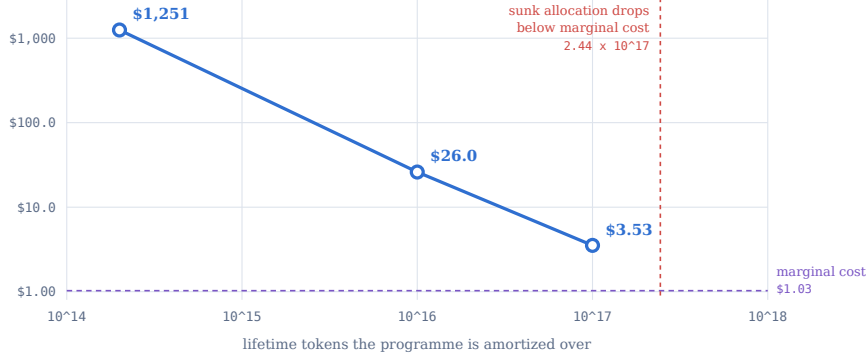


Figure 2: The cost stack of one 10^6 -token result on a log axis: electricity 28/375, depreciation 625/657, marginal 84257/82125, programme allocation 1250, total 102740507/82125. Every bar is the exact rational proved in `RequestProject/Ledger.lean`.

The answer is an amortization assumption

Fully loaded cost of the same result on the same cluster, against the lifetime token volume the programme is spread over. The physics never changes.

cost of one result



Ledger.lean · lifetime_assumption_dominates, amortized_le_marginal_of_large_lifetime

Figure 3: The same result against the deployed lifetime volume the frontier programme is amortized over: \$1251.03 at 2×10^{14} , \$26.03 at 10^{16} , \$3.53 at 10^{17} , against a marginal-cost floor of $84257/82125 \approx \$1.03$, with the crossover at 2.44×10^{17} tokens.

3 The cost ledger

The ledger is one chain of exact rational arithmetic, and each arrow in it is a definition in RequestProject/Theory/WorkloadCost.lean applied to the parameters of Data/Scenarios.lean:

$$\begin{array}{lll}
 \text{cluster power} \times \text{run time} & \longrightarrow & \text{energy of the run} \\
 \text{energy} \rightarrow \text{kWh} \rightarrow \text{dollars} & \longrightarrow & \text{electricity} \\
 \text{cluster capital} \times \text{occupancy} & \longrightarrow & \text{depreciation} \\
 \text{programme cost} \div \text{lifetime volume} & \longrightarrow & \text{allocation per token} \\
 & \longrightarrow & \text{cost per result, cost per token.}
 \end{array}$$

Three prices are kept apart throughout, and their relationship is an identity rather than a convention:

$$\underbrace{\text{electricity} + \text{depreciation}}_{\text{marginal}} + \underbrace{\frac{C_{\text{programme}}}{L_{\text{lifetime}}} \cdot T_{\text{task}}}_{\text{amortized replacement}} = \text{fully loaded.}$$

The identity is Tokenomics.fullyAmortized_decomposition in Theory/AITokenomics.lean, stated for arbitrary parameters; the ledger’s instance of it is marginalUSD_eq, amortizedUSD_base and ledgerTotal_base. Collapsing the three into one number is exactly the move the development refuses to make: the marginal cost is what one more result costs on an existing cluster, the replacement allocation is a capital convention, and only their sum is the “cost” that a replacement-cost accountant would recognise.

Two boundary conditions are proved rather than assumed.

Proposition 2 (total_is_lower_bound). *Human review, problem selection and the verification pipeline are excluded from the ledger. For any overhead parameters with nonnegative visible cost and multiplier, the ledger total is at most the total with that overhead included. Every figure in Table 1 is therefore a lower bound on the all-in cost.*

Proposition 3 (marginal_le_ledgerTotal). *The marginal cost of a run is at most its fully loaded cost, for every nonnegative parameter set: no accounting boundary that ignores the programme can overstate the result.*

Remark 4 (Why the denominators are guarded). *Every quantity in the ledger is a quotient, and a parameter set with a zero lifetime volume would silently price the programme at nothing. The development therefore proves the denominators positive (`zkLedger_denominators_pos`) and states well-formedness as a predicate (`Theory/Invariants.lean`) rather than trusting the reader to supply sensible numbers.*

4 The Bitcoin counterfactual

The comparison the ledger supports is between one accepted result and the mining the same resources would have bought. It is deliberately given twice, because “the same resources” means two different things.

The mining side is not a constant fetched from a website; it is derived from network parameters in `RequestProject/Theory/Mining.lean`:

$$\text{network power} = H \cdot e_{\text{hash}}, \quad E_{\text{block}} = \text{network power} \cdot t_{\text{block}}, \quad E_{\text{coin}} = \frac{E_{\text{block}}}{\text{coins per block}}.$$

At the recorded network — 10^{21} H/s at 20 J per terahash, a ten-minute block, 3.125 coins issued per block — this gives $E_{\text{coin}} = 3.84 \times 10^{12}$ J, and at \$0.08/kWh an electricity-only cost of about \$85 333 per coin. The allocation convention is visible at every step: the energy of a coin is the energy of a block divided by an explicit coins-per-block parameter, so a halving doubles it (`energyPerCoinJ_halving`), and a rise in network hashrate raises it (`energyPerCoinJ_mono_hashrate`).

Definition 5 (The two counterfactuals). *`coinsFromEnergy(n, E) = E / Ecoin(n)` is what the run’s own joules would have mined. `coinsFromMoney(n, p, B) = B / (electricity cost per coin at p)` is what the run’s own dollars would have mined, when the only mining cost counted is electricity at the same tariff.*

Theorem 6 (`claim_btc_from_same_energy, claim_btc_from_same_money`). *At the recorded parameters, the 3 360 000 J of the run buy 7/8000000 BTC = 87.5 satoshi; the \$1251.03 charged to the run buy 102740507/7008000000 BTC $\approx$ 1 466 046 satoshi.*

Theorem 7 (`claim_money_beats_energy_counterfactual`). *The money counterfactual exceeds the energy counterfactual by more than four orders of magnitude: $10^4 \cdot \text{coinsFromEnergy}(\dots) < \text{coinsFromMoney}(\dots)$.*

The gap is the whole point of stating both. Physically, an accepted result is cheap: a kilowatt hour of grid energy, less than a tenth of a dollar of electricity, four orders of magnitude less than a Bitcoin block. Financially it is not, because almost all of the money charged to it is allocated capital that was spent before the run started and would have been spent had the run never happened. An equal-energy comparison measures the physics; an equal-money comparison measures the amortization convention. Quoting one of them as “the” comparison is where such arguments usually go wrong, and it is why the development proves both and their inequality.

Remark 8 (What the mining side omits). *The money counterfactual counts electricity only. ASIC capital, hosting, difficulty adjustment, block-reward schedule changes and the fact that deploying enough hardware would move the network’s own competitive conditions are not modelled in the counterfactual figure — although the rig-level model (`MiningRig, costPerCoinUSD, breakevenElectricityUSDPerKWh`) does carry capital and lifetime, and proves the break-even tariff condition (`profitable_iff_price_le_breakeven`). Read the BTC figures as the electricity-equivalent counterfactual they are.*

5 The protocol-layer obligations

Throughout, F is a field of characteristic zero (the wire field), G is the commitment group, and q is its prime order. `PolySystem F` is a list of polynomial expressions asserted to vanish; `GateCircuit F` is a list of flat gates; `Sat` is satisfaction of either by an assignment $\sigma : \mathbb{N} \rightarrow F$.

§2 The semantic bridge

Definition 9 (bridge). *For every core term tm and type T with $tm.infer = some\ T$: the arithmetic relation `CoreSat  $tm$` holds at the wire word `CTy.wires F T`, and at no other wire word; and any wire word at which it holds witnesses that tm really evaluates to a value of type T .*

This is the requirement that stops the system from proving only that some constraint system was satisfied. Discharged by `ZkPoP.Spec.declaration_bridge`, which is in turn `Core.coreCircuitRel_sound` (satisfaction entails evaluation and well-typedness) together with `coreCircuitRel_complete` (the honest encoding satisfies the relation), so the relation is pinned from both sides.

Definition 10 (conversion). *Definitionally equal types — equal after evaluation of index expressions — receive one and the same wire word: `CTy.DefEq T U  $\rightarrow$  CTy.wires F T = CTy.wires F U`.*

Discharged by `Core.CTy.flat_norm`. The converse, `wires_injective_on_norm`, says that only definitionally equal types do, so the encoding neither conflates nor splits.

§3 Arithmetization

Definition 11 (index_nat). *Every entry of every emitted wire word is a natural number embedded in the field: $x \in CTy.wires\ F\ T \rightarrow IsNatF\ F\ x$.*

Definition 12 (compilation). *For a polynomial system sys all of whose wires lie below the scratch counter n , an assignment σ satisfies sys if and only if σ extends to an assignment satisfying the compiled circuit `flatSys sys n`.*

The forward direction is the interesting one: it rules out junk assignments of the scratch wires introduced by flattening. Discharged by `ZkPoP.flatSys_sat_iff`.

Definition 13 (range_sound, range_complete). *If the bit-decomposition system `rangeSys x bs` is satisfied then the checked wire carries a natural number below $2^{|bs|}$ (`IsNatBounded`); and conversely every natural number below 2^k admits a satisfying bit assignment.*

Soundness and completeness together are what makes the range check a check rather than a filter. The reason it is needed at all is itself a theorem: `Range.isNatF_of_zmod` shows `IsNatF` is vacuous in $\mathbb{Z}/p$, so in a prime field the useful predicate is the *bounded* one, and it must be enforced by constraints.

Definition 14 (self_hosting). *For every circuit C and witness u , the universal host `hostCircuit C u` satisfies four conditions at once: it is satisfiable exactly when `C.Sat u`; hosting a host still certifies exactly the inner claim; every satisfying assignment carries C 's own code word on the wires above `C.bound`; and its size is `entryHostSize C = C.bound + 5 * C.length`.*

The third condition is what makes the recursion *self-describing*: the code word exposed by a satisfying assignment determines the hosted circuit up to normalisation (`Encoding.code_injective`, `host_self_describing`). The fourth is the price of recursion, counted rather than asserted; on a one-gate circuit the kernel evaluates it to $1 \mapsto 8 \mapsto 47$ gates (`EncodingDemo.host_of_host_size`).

§4 Commitment and Σ -protocol

Definition 15 (`commitment_homomorphic`, `sigma_complete`, `sigma_extract`, `sigma_zk`). *Pedersen commitment $\text{commit}(g, h, m, r) = m \cdot g + r \cdot h$ is additively homomorphic; the honest prover always convinces the verifier of $z \cdot g + w \cdot h = t + c \cdot C$; two accepted transcripts sharing an announcement but with distinct challenges yield, via `Schnorr.extract`, an opening of the published commitment; and over the whole randomness space the honest transcripts and the simulated ones are the same multiset.*

Extraction is unconditional as extraction: it produces a pair (m, r) opening C . Reading it as “the prover knew the witness” — and reading commitment as binding — is where the discrete logarithm assumption enters, and it enters as an assumption (§10).

§5 The shard layer

Definition 16 (`shard_privacy`). *Systematic erasure encoding fails the privacy audit outright; the randomized (ramp) encoding passes it; and the secret is still recovered from two nodes with distinct evaluation points.*

The first conjunct is a negative result and is the specification’s reason for mandating randomized encoding: it is proved, not assumed, via `Shard.deterministic_not_private`.

§6–§7 The artifact and its decoder

Definition 17 (`artifact`). *For a deployment D and a catalogue index i in range, the published triptych `publish D F i m r r1 r2` is *Valid* at i ; and, given that the Fiat–Shamir hash is injective in its digest argument and that the statement digest is injective, no valid triptych carrying the same `QR_COMMIT` and `QR_TRANSCRIPT` can be read as a different catalogue entry.*

Definition 18 (`decoder`). *The three-step offline verifier — field bounds, both sides of the verification equation, equality — returns `true` on the honest triptych, run on the integer representatives that a QR code actually carries.*

The bounds step is load-bearing rather than cosmetic: an unreduced representative can satisfy the group equation while lying outside the field (`Decoder.unreduced_still_satisfies_equation`, `zk_decoder_rejects_unreduced`).

6 The application-layer obligations

zkPoP is applied in this project to a workload whose rounds are certified individually and chained: the closing wires of round t are the opening wires of round $t + 1$, so no round can be skipped, replayed or rewritten. What the conformance predicate records at this layer is the cost side — the part that connects a proof to the economics of §3.

Definition 19 (`AppObligations`). *One round of one seat is 106 gates. A match of S seats over T rounds is $106ST$ gates. Its verification cost is that gate count at the gate price. The energy conversion the cost model uses, $E_J = E_{\text{kWh}} \times 3,600,000$, is a theorem. When the gate price is the price of the gate’s electricity, the cost of a proof is its energy in kilowatt hours times the tariff. Cost is monotone in the number of rounds. And the marginal cost of a round, per seat, is the constant $106 \times (\text{price per gate})$.*

The last field deserves a word. Because the gate count of a round does not depend on the data of that round, the artifact is the same size whatever it certifies: a losing round costs exactly what a winning one costs. Size obliviousness is a privacy property, and here it is an arithmetic identity — `Game.GateCost.gates_per_round_constant`.

Every physical and monetary quantity in this layer (`joulesPerGate`, `usdPerGate`, the tariff) is a parameter. No empirical figure is asserted by any theorem; the theorems are the accounting identities and the comparative statics that follow from whatever figures are supplied.

7 The conformance theorem and the paper certificate

Theorem 20 (`ZkPoP.Conformance.zkpop_conformance`). *For every characteristic-zero field F , every commitment group G that is a module over $\mathbb{Z}/q$ with q prime,*

$$\text{Obligations } F \ G \ q \wedge \text{AppObligations}.$$

Proof (as machine-checked). By the anonymous constructor, from `project_conforms` and `application_conforms`. Each field of `Obligations` is discharged by the theorem named opposite it in Table 2, and each field of `AppObligations` by the theorem named in Table 3. The substance of the proof is in those theorems; the content of *this* theorem is that the obligations, as stated, are exactly what they discharge — which is checked by the elaborator, not by a reader. $\square$

Theorem 21 (`ZkPoP.Paper.paper_certificate`). *Under the same hypotheses,*

$$\text{Obligations } F \ G \ q \wedge \text{AppObligations} \wedge \text{LedgerFindings} \wedge \text{PublishedEvidence}.$$

`LedgerFindings` has eighteen fields: the ten certified values of Table 1 (the energy-per-token row is certified separately, by `energyPerToken_eq`), four dominance and sensitivity statements, two scaling laws for discarded search, the lower-bound statement, and the mining halving law. `PublishedEvidence` has six: the committed digits are the ledger’s inputs, the denominators are positive, the commitment is the Pedersen commitment to the packed vector, the opening is unique, the transcript verifies, and the simulation verifies. Both are proved in `RequestProject/ZkPoPPaper.lean`, and every field is discharged by a named theorem of the development — so the paper and the theorem cannot drift apart without the build breaking.

Table 2: Protocol-layer obligations and the theorems that discharge them.

Obligation	Discharged by
<code>bridge</code>	<code>Spec.declaration_bridge</code> $\leftarrow$ <code>Core.coreCircuitRel_sound</code> , <code>coreCircuitRel_complete</code>
<code>conversion</code>	<code>Core.CTy.flat_norm</code>
<code>index_nat</code>	<code>Core.CTy.wires_isNatF</code>
<code>compilation</code>	<code>flatSys_sat_iff</code> $\leftarrow$ <code>Poly.flat_out_eval</code> , <code>Poly.fill_gates</code>
<code>range_sound</code>	<code>Range.rangeSys_isNatBounded</code>
<code>range_complete</code>	<code>Range.rangeSys_complete</code>
<code>self_hosting</code>	<code>Spec.self_hosting</code> $\leftarrow$ <code>Encoding.host_sat_iff</code> , <code>host_two_level</code> , <code>host_exposes_code</code> , <code>hostCircuit_length</code>
<code>commitment_homomorphic</code>	<code>Pedersen.pedersen_add</code>
<code>sigma_complete</code>	<code>Schnorr.completeness</code>
<code>sigma_extract</code>	<code>Schnorr.special_soundness</code>
<code>sigma_zk</code>	<code>Schnorr.zero_knowledge</code>
<code>shard_privacy</code>	<code>Spec.privacy_audit</code> $\leftarrow$ <code>Shard.erasure_not_private</code> , <code>ramp_</code> <code>private</code> , <code>ramp_recover</code>
<code>artifact</code>	<code>Spec.artifact_valid_and_binding</code> $\leftarrow$ <code>Artifact.publish_valid</code> , <code>statement_binding</code>

Obligation	Discharged by
decoder	<code>Spec.decoder_accepts_published</code> $\leftarrow$ <code>Artifact.decoder_correct</code>

Table 3: Application-layer obligations and the theorems that discharge them.

Obligation	Discharged by
round_size	<code>Game.roundSys_gates_uncapped</code>
match_size	<code>Game.matchSys_gates</code>
match_cost	<code>Game.GateCost.matchCost</code>
unit_conversion	<code>Game.GateCost.energyJ_eq_kWh</code>
electricity	<code>Game.GateCost.ofElectricity_costUSD</code>
cost_mono	<code>Game.GateCost.matchCost_mono</code>
size_oblivious	<code>Game.GateCost.gates_per_round_constant</code>

Table 4: Findings and evidence: the fields of `LedgerFindings` and `PublishedEvidence`, and the theorems that discharge them.

Field	Discharged by
energy_joules	<code>Ledger.resultEnergyJ_eq</code>
energy_kWh	<code>Ledger.resultEnergyKWh_eq</code>
electricity	<code>Ledger.electricityUSD_eq</code>
depreciation	<code>Ledger.depreciationUSD_eq</code>
marginal	<code>Ledger.marginalUSD_eq</code>
allocation	<code>Ledger.amortizedUSD_base</code>
total_investment	<code>Ledger.ledgerTotal_base</code>
cost_per_token	<code>Ledger.ledgerCostPerToken_base</code>
sunk_dominates_marginal	<code>Ledger.sunk_cost_dominates_marginal</code>
btc_from_same_energy	<code>ZkLedger.claim_btc_from_same_energy</code>
btc_from_same_money	<code>ZkLedger.claim_btc_from_same_money</code>
money_counterfactual_dominates	<code>ZkLedger.claim_money_beats_energy_counterfactual</code>
allocation_antitone_lifetime	<code>WorkloadCost.training_cost_scale_monotonicity</code>
lifetime_assumption_dominates	<code>Ledger.lifetime_assumption_dominates</code>
discovery_scales	<code>Ledger.ledgerTotal_scales</code>
cost_per_token_scale_invariant	<code>Ledger.ledgerCostPerToken_scale_invariant</code>
total_is_lower_bound	<code>Ledger.total_is_lower_bound</code>
halving_doubles_energy	<code>Mining.energyPerCoinJ_halving</code>
inputs_are_committed	<code>ZkLedger.committed_inputs_are_ledger_inputs</code>
denominators_positive	<code>ZkLedger.zkLedger_denominators_pos</code>
commitment_correct	<code>ZkLedger.zkCommitment_eq</code>
inputs_unique	<code>ZkLedger.committed_vector_unique</code>
transcript_verifies	<code>ZkLedger.zkTranscript_verifies</code>
simulation_verifies	<code>ZkLedger.zkSimulated_verifies</code>

8 The published proof: sealing the inputs

A cost model can always be made to say anything by choosing its inputs afterwards. The purpose of the published artifact is to remove that freedom. The eight inputs of the ledger are rescaled to integers, packed positionally in base 2^{31} , and committed to with a Pedersen commitment in a 2048-bit Schnorr group; the prover then runs a two-generator Σ -protocol and publishes the transcript.

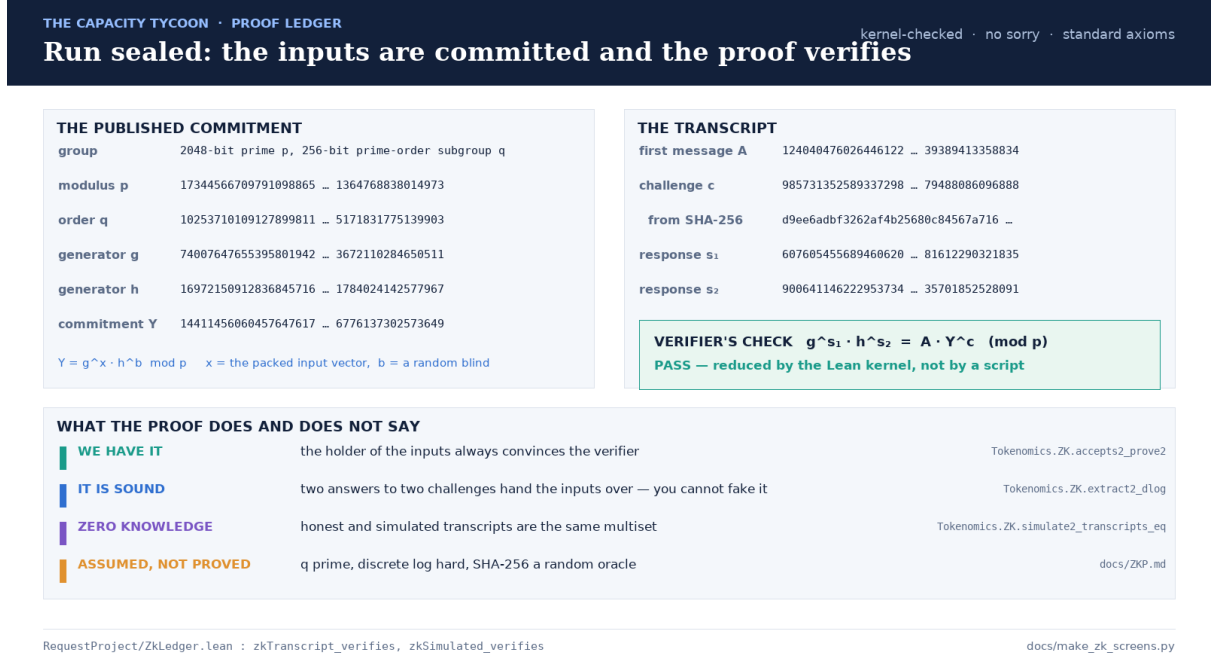


Figure 4: The published commitment and the recorded transcript. The verification equation $g^{s_1}h^{s_2} = A \cdot Y^c$ is reduced by the Lean kernel on 2048-bit numerals, not by a script (ZkLedger.zkTranscript_verifies).

THE CAPACITY TYCOON · INPUT AUDIT

each digit is an input of the certified ledger

The eight digits under the seal, and what they are

THE COMMITTED INPUT VECTOR				
#	FIELD	VALUE	UNIT	LEDGER NAME
0	frontier_programme_kUSD	250 000 000	thousand USD	frontierProgramUSD / 1000
1	lifetime_tokens_millions	200 000 000	million tokens	lifetimeTokensBase / 10 ⁶
2	result_tokens	1 000 000	tokens	resultTokens
3	cluster_purchase_USD	2 400 000	USD	clusterPurchaseUSD
4	cluster_IT_watts	56 000	watts	(cluster _).itWatts
5	tokens_per_second	20 000	tokens/s	tokensPerSecond
6	electricity_microUSD_per_kWh	80 000	μUSD/kWh	electricityUSDPerKWh × 10 ⁶
7	pue_milli	1 200	thousandths	pue × 1000

packed, base 2³¹, little-endian → x = 2527495078509144182217 ... 911873838921462400
x < q, so nothing wraps: the digits can be read back off the packed number

PROVED HERE

- ✓ every digit positive and below the radix
- ✓ the digits are the ledger's own inputs, in the units named
- ✓ one commitment ⇒ one input vector (packing is injective)

NOT PROVED HERE

- that the inputs describe any real company
- that the modelling assumptions are the right ones
- the data layer labels every one of them an assumption

RequestProject/ZkLedger.lean : zkInputs_wellFormed, committed_inputs_are_ledger_inputs
docs/make_zk_screens.py

Figure 5: The eight committed digits and the ledger name of each, together with what the seal does and does not establish (`ZkLedger.committed_inputs_are_ledger_inputs`, `zkInputs_wellFormed`).

Four theorems make the seal mean something.

1. **The digits are the ledger's inputs** (`committed_inputs_are_ledger_inputs`), field by field, in stated units: the programme in thousands of dollars, the lifetime volume in millions of tokens, the result in tokens, the cluster's price in dollars and its draw in watts, throughput in tokens/second, electricity in micro-dollars per kWh, PUE in thousandths.
2. **The commitment admits one input vector** (`committed_vector_unique`). Packing is injective below the group order (`zkWidth_fits`, `zkWitness_lt_order`), so any other eight-digit vector of the same width opening to the published element *is* the committed one — stated for an arbitrary prime-order group with a generator.
3. **The transcript verifies** (`zkTranscript_verifies`), and it verifies *as an instance of completeness*: the responses are shown to be the honest responses and the equation then follows from `acceptsExp2_prove`, rather than from a numerical coincidence.
4. **A simulated transcript verifies too** (`zkSimulated_verifies`), produced with no witness whatsoever: challenge and responses chosen first, the announcement solved for afterwards. So an accepted transcript, on its own, discloses nothing — which is what makes publishing it safe, and what makes the commitment, not the transcript, the thing that binds.

9 The audit: proof that these are proofs

Four checks make the certificate worth the name.

No holes. No file in `RequestProject/` contains `sorry` — 264 Lean files, 79 825 lines, checked by `rg` over the tree. In particular the `zkPoP` layer (`RequestProject/ZkPoP/*.lean`, `ZkPoPSkill1.lean`, `ZkPoPSpec.lean`, `ZkPoPDecoder.lean`, `ZkPoPVectors.lean`, `ZkPoPConformance.lean`) and the

ledger layer (`Ledger.lean`, `ZkLedger.lean`, `Theory/AITokenomics.lean`, `Theory/Mining.lean`, `Theory/WorkloadCost.lean`) are complete.

No extra axioms. `ZkPoPConformance.lean` and `ZkPoPPaper.lean` print the axiom closure of `project_conforms`, `application_conforms`, `zkpop_conformance`, `ledger_findings`, `published_evidence` and `paper_certificate` as part of the build. Each reports only `propext`, `Classical.choice` and `Quot.sound`. There are no axiom declarations, no `@[implemented_by]` attributes and no `native_decide` in this layer: the concrete evaluations — gate counts, decoder verdicts, the $1 \mapsto 8 \mapsto 47$ hosting sizes, and the 2048-bit modular exponentiations of the transcript — are closed by ordinary kernel computation.

It builds. `lake build` completes with zero errors over the whole tree (8 290 jobs in the recorded run). The certificate module alone is `lake build RequestProject.ZkPoPPaper`.

Nothing vacuous. The obligations quantify over arbitrary parameters, so no field can be satisfied by a convenient choice of instance; the findings are equalities between explicit rationals, which cannot hold vacuously; and the machinery is exercised on concrete data — `SelfHostDemo` shows the universal circuit accepting a valid assignment and rejecting an invalid one, `Vectors.demo_accept` and `demo_reject` give the decoder verdicts by `decide`, and `Decoder.zk_decoder_accepts` runs the decoder on a real 2048-bit Schnorr group.

The figures agree with the theorems. The drawing scripts contain no economics: the certified rationals are hard-coded at the top of `docs/make_figures.py` beside the name of the backing theorem, and `scripts/check_constants.py` requires every exact rational written in a drawing script, in `README.md` or in `docs/FIGURES.md` to occur, in lowest terms, in some Lean source. It exits nonzero on any disagreement.

10 What is not claimed

Delimiting the certificate is part of it. The following are *not* theorems of this development, and the obligations and findings above are stated so that none of them is silently used.

Cryptographic

1. **Collision-freeness is a hypothesis.** The binding half of `artifact` quantifies over injectivity of the Fiat–Shamir hash in its digest argument and injectivity of the statement digest. No concrete hash is modelled, and for a fixed finite hash no such property can be proved unconditionally.
2. **Binding rests on discrete logarithms.** `Pedersen.binding_dlog` is a reduction: it is stated relative to knowledge of $\log_h g$. Hardness in a particular group is neither proved nor provable here. Hiding, by contrast, is unconditional.
3. **The published group’s order is assumed prime.** The kernel checks $g^q = 1$ and $g \neq 1$; primality of q and the random-oracle behaviour of the SHA-256 challenge are recorded assumptions.
4. **IsNatF is vacuous in $\mathbb{Z}/p$,** by `Range.isNatF_of_zmod`. This is why the substantive obligation is the bounded one, enforced by `rangeSys`.
5. **Zero knowledge is distributional.** The statement is identity of transcript multisets, not simulation against an arbitrary adversary with auxiliary input.

6. **The catalogue digest and the Merkle compression function are toy functions.** $\text{hashF}(x, y) = (x + y)^3 + x$ is a polynomial chosen so that it can live inside a constraint system; no collision resistance is claimed.
7. **entryHostSize is a gate count, not a proving cost.** No polynomial commitment scheme, proof size or prover runtime is modelled.
8. **The published parameter set is a worked example.** The 2048-bit group is real and the transcript verifies, but its witness is published in the repository; it is a demonstration, not a secret.
9. **The WebAssembly decoder is derived, not verified.** `gatecheck.wasm` is assembled from the committed `gatecheck.wat`, whose intended semantics is `GateCircuit.check`; the correspondence is established by cross-language conformance testing against Lean-derived vectors, not by a proof about WebAssembly.
10. **The object language is small by design.** The bridge is a real bridge for the core language of `ZkPoP/Core.lean`, not for Lean itself.

Economic

11. **No empirical figure is a theorem.** The cluster, the tariff, the throughput, the PUE, the \$250B programme, the 2×10^{14} lifetime volume and the mining network are *data*. The data layer labels each of them an assumption; the theorems say what follows from them. Replacing a figure changes the certified value and nothing else in the argument.
12. **The programme allocation is a convention, not a measurement.** Charging \$1250 of a \$250B programme to one result is the arithmetic of dividing by an assumed lifetime volume. The sensitivity results exist precisely because that divisor, not the physics, determines the headline.
13. **Human cost is excluded, and its exclusion is proved to be downward.** Problem selection, expert review and the verification pipeline are not priced; `total_is_lower_bound` says every reported total is a lower bound on the all-in cost. In an honest ledger for a real result, the human line would very plausibly dominate everything in Table 1.
14. **The token count of a result is an assumption.** 10^6 effective tokens for an accepted result and a hundredfold discovery multiplier for the campaign are stipulated; the linear scaling laws are what make those stipulations easy to change.
15. **The Bitcoin counterfactual counts electricity only.** ASIC capital, hosting, difficulty adjustment and the reward schedule are outside the counterfactual figure, and deploying capital at that scale would move the network itself.
16. **Nothing here is a claim about any company's accounts.** The ledger is a model of a stated scenario, not a reconstruction of anyone's financial statements, and no result of this project should be read as an assertion about what a particular laboratory spent.

11 Reading the artifact

Paths below are relative to the repository root; `ZkPoP/` abbreviates `RequestProject/ZkPoP/` and unqualified `*.lean` files sit in `RequestProject/`.

File	Contents
ZkPoPPaper.lean	the certificate behind this paper: findings, evidence, conjunction
ZkPoPConformance.lean	the obligations and the conformance theorem
ZkPoPSpec.lean	the specification, section by section
ZkPoP/Core.lean, CoreCircuit.lean	core language, checker, semantic bridge
ZkPoP/Circuit.lean	polynomial systems, gates, flattening
ZkPoP/Catalog.lean	the verified gadget catalogue
ZkPoP/Range.lean	bit-decomposition range checks
ZkPoP/Encoding.lean	code words, the universal host, its size
ZkPoP/Binding.lean	Pedersen commitments, the Σ -protocol
ZkPoP/Shards.lean, ShardPrivacy.lean	threshold distribution and the privacy audit
ZkPoP/Artifact.lean	the triptych, validity, binding, the decoder
ZkPoP/GameRound.lean, GameTrace.lean	the application: rounds, traces, gate cost
Ledger.lean	the certified cost ledger of one result
ZkLedger.lean	the sealed inputs, the transcript, the four headline claims
Theory/WorkloadCost.lean	marginal, amortized and fully loaded cost
Theory/AITokenomics.lean	the abstract per-result cost model and its identities
Theory/Mining.lean	the proof-of-work network and rig models
Data/Scenarios.lean, Data/ZooScenarios.lean	the replaceable empirical layer
docs/ZKPOP.md, docs/ZKPOP_SPEC.md	the section-by-section maps
docs/FIGURES.md, docs/SCREENS.md	what each figure plots and which theorem certifies it

The whole development is checked by `lake build`; the certificate module alone is

```
lake build RequestProject.ZkPoPPaper.
```